Genesis
Análisis preventivo de clientes

Genesis — Sistema de análisis preventivo de clientes

Documentación del proyecto de Machine Learning

Aplicación web: Genesis, desarrollada con Python y Streamlit

Curso: Machine Learning

Institución: [Escriba aquí el nombre de la universidad]

Docente: [Escriba aquí el nombre del docente]

Fecha: Octubre de 2026

1. Integrantes del grupo

Complete esta tabla con los datos de cada integrante antes de entregar el documento.

N.°	Nombre completo	Código o documento
1	Oscar Andres Sanchez Caballero	[Complete]
2	[Nombre completo]	[Complete]
3	[Nombre completo]	[Complete]
4	[Nombre completo]	[Complete]

2. Introducción y problema que resuelve

Las empresas de servicios dependen de conservar a sus clientes. Captar un cliente nuevo cuesta más que mantener uno existente, y muchas veces la empresa se entera de que el cliente se fue cuando ya es tarde.

Genesis es una aplicación web que ayuda al equipo a identificar a tiempo a los clientes que podrían requerir seguimiento. A partir de datos históricos de la empresa, un modelo de clasificación estima la probabilidad de que un cliente abandone y la traduce en un nivel de atención (bajo, medio o alto) acompañado de una recomendación sencilla. Cualquier persona puede ingresar los datos de un cliente y obtener una evaluación clara, visual y comprensible, sin necesidad de conocer Machine Learning.

3. Objetivos

Objetivo general

Desarrollar una aplicación web que utilice un modelo de clasificación entrenado con datos históricos para estimar el riesgo de abandono de clientes de una empresa de servicios y presentar el resultado de forma clara.

Objetivos específicos

- Analizar y preparar con Pandas un conjunto de datos de clientes con al menos 150 registros.
- Entrenar y comparar al menos dos modelos de clasificación usando accuracy, matriz de confusión y reporte de clasificación.
- Seleccionar el modelo más adecuado para una aplicación preventiva y justificar la decisión.
- Construir una interfaz profesional y responsive con panel principal, formulario, resultado visual, gráficas e historial.
- Registrar cada evaluación en una base de datos para consultarla, filtrarla y exportarla.
- Documentar el proceso, los resultados y las limitaciones del proyecto.

4. Tecnologías utilizadas

Tecnología	Uso en el proyecto
Python	Lenguaje principal de todo el proyecto, tanto del modelo como de la interfaz.
Streamlit	Construcción de la aplicación web, formulario, navegación y páginas.
Pandas y NumPy	Exploración, limpieza y preparación de los datos.
scikit-learn	Pipeline, escalado, modelos de clasificación, métricas y validación cruzada.
joblib	Guardado y carga del modelo entrenado.
Plotly	Gráficas interactivas del panel y del análisis técnico.
SQLite	Base de datos del historial de evaluaciones.
CSS dentro de Streamlit	Diseño visual, tarjetas, barra de progreso y adaptación a celular.
Git y GitHub	Control de versiones y publicación del código.
Streamlit Community Cloud	Despliegue de la aplicación.

5. Descripción del dataset

El conjunto de datos tiene 505 filas originales y 500 registros después de la limpieza. Cada fila es un cliente. La variable objetivo es abandono: vale 1 si el cliente se fue y 0 si permanece. En la base limpia, 176 clientes abandonaron (35,2 %) y 324 permanecen.

Variable	Descripción	Mínimo, promedio y máximo
edad	Edad del cliente en años	18, 37,6, 75
ingresos	Ingresos mensuales en pesos colombianos	$1.000.000, $3.579.300, $11.350.000
frecuencia_compra	Compras realizadas por mes	0, 3,7, 13
cantidad_productos	Productos adquiridos en los últimos 12 meses	1, 7,5, 28
tiempo_cliente	Meses como cliente	1, 31,0, 120
satisfaccion	Nivel de satisfacción de 1 a 10	1, 6,5, 10
abandono	Variable objetivo: 1 abandona, 0 permanece	0 o 1

Origen de los datos. Los datos son sintéticos. El script generar_dataset.py los crea con una regla de negocio y ruido aleatorio: el abandono es más probable cuando la satisfacción, la antigüedad y la frecuencia de compra son bajas. No son clientes reales.

6. Proceso de limpieza y preparación de datos

El archivo original (data/clientes_raw.csv) se creó con problemas a propósito para mostrar el proceso de limpieza. El script entrenar.py los trata con Pandas:

1. Se eliminaron 5 filas duplicadas (505 a 500 filas).

2. Se marcaron como nulas 2 edades imposibles (menores de 18 o mayores de 100) y 1 valor de satisfacción fuera del rango de 1 a 10.
3. Se imputaron con la mediana los 15 valores nulos de las variables de entrada (12 originales y 3 provenientes del paso anterior). La mediana se eligió porque no se distorsiona con valores extremos.
4. Se convirtieron a entero las columnas edad, ingresos, tiempo como cliente y satisfacción.
5. Se verificó con una comprobación automática que no quedaran nulos y se guardó el resultado en data/clientes.csv.

7. Definición de X e y

X (variables de entrada, 6 columnas): edad, ingresos, frecuencia_compra, cantidad_productos, tiempo_cliente y satisfaccion.

y (variable objetivo): abandono, con 1 para el cliente que se fue y 0 para el que permanece.

No se usan el identificador ni el nombre del cliente como entradas, porque no aportan información sobre su comportamiento.

8. División de entrenamiento y prueba

Los 500 clientes se dividieron en 400 para entrenar (80 %) y 100 para probar (20 %), con semilla 42 para que el resultado se pueda repetir. La división es estratificada: ambos grupos conservan la misma proporción de abandonos (141 abandonos en entrenamiento y 35 en prueba). El modelo nunca ve los datos de prueba durante el entrenamiento.

9. Modelos entrenados y comparación de resultados

Se entrenaron cuatro modelos de clasificación con los mismos datos. La regresión logística y KNN usan un escalador (StandardScaler) dentro de un Pipeline, de modo que la aplicación transforma los datos de la misma forma que en el entrenamiento. Precisión, recall y F1 se calculan para la clase Abandona.

Modelo	Accuracy	Precisión	Recall	F1	Accuracy val. cruzada	Recall val. cruzada
Regresión logística (estándar)	75,0 %	72,7 %	45,7 %	0,561	78,3 %	63,2 %
Regresión logística (balanceada) (en uso)	**73,0 %**	**61,1 %**	**62,9 %**	**0,620**	**80,3 %**	**79,5 %**
Random Forest	73,0 %	68,2 %	42,9 %	0,526	76,8 %	59,7 %
KNN (vecinos más cercanos)	74,0 %	68,0 %	48,6 %	0,567	75,3 %	63,8 %

La validación cruzada repite el entrenamiento y la prueba cinco veces con los datos de entrenamiento. Con solo 100 clientes de prueba, cada error mueve el accuracy un punto, así que la validación cruzada da una estimación más estable.

Matriz de confusión del modelo en uso

Las filas son lo que ocurrió en realidad y las columnas lo que predijo el modelo, sobre los clientes de prueba.

	Predicho: Permanece	Predicho: Abandona
Real: Permanece	51 (aciertos)	14 (alertas innecesarias)
Real: Abandona	13 (abandonos sin detectar)	22 (abandonos detectados)

Reporte de clasificación del modelo en uso

Clase	Precisión	Recall	F1
Permanece	0,80	0,78	0,79
Abandona	0,61	0,63	0,62
Promedio simple	0,70	0,71	0,71

10. Modelo seleccionado y justificación

Modelo seleccionado: Regresión logística (balanceada), con 73,0 % de accuracy en prueba y 62,9 % de recall en la clase Abandona.

Genesis es una aplicación preventiva: su propósito es avisar a tiempo cuando un cliente podría irse. Un modelo puede equivocarse de dos maneras: no detectar a un cliente que sí se va, o dar una alerta innecesaria sobre uno que se queda. Perder a un cliente suele costar más que una llamada de seguimiento de más, por eso se dio prioridad a detectar abandonos sin sacrificar demasiado el accuracy.

La regresión logística estándar tenía el mayor accuracy, pero detectaba menos de la mitad de los abandonos. Se evaluó la misma regresión con class_weight="balanced", que hace que el modelo preste más atención a la clase minoritaria (los clientes que abandonan). La comparación fue esta:

Resultado en prueba	Estándar	Balanceada
Accuracy	75,0 %	73,0 %
Abandonos detectados (recall)	16 de 35 (46 %)	22 de 35 (63 %)
Abandonos sin detectar	19	13
Alertas innecesarias	6	14
Precisión de las alertas	73 %	61 %
F1 de la clase Abandona	0,561	0,620
Recall en validación cruzada	63 %	79 %

Decisión. La versión balanceada detecta 22 de los 35 clientes que abandonaron, frente a 16 de la versión estándar. A cambio, el accuracy baja 2 puntos y aparecen 8 alertas innecesarias más. Para una aplicación de retención ese intercambio es razonable, por lo que Genesis usa la versión balanceada.

Regla de selección. Entre los modelos cuyo accuracy en prueba queda a menos de 3 puntos del mejor, se elige el de mayor F1 de la clase Abandona, porque equilibra detectar abandonos (recall) y evitar alertas innecesarias (precisión).

Las métricas de los modelos originales no cambiaron al agregar la versión balanceada: se entrenaron con los mismos datos y la misma división.

11. Explicación de las métricas

Métrica	Qué significa
Accuracy	De cada 100 clientes, cuántos clasifica bien el modelo, tanto los que se quedan como los que se van. En este modelo: 73 de 100.
Matriz de confusión	Tabla que cuenta los aciertos y los errores del modelo, separando los dos tipos de error: abandonos que no detectó y alertas que no eran necesarias.
Precisión	Cuando el modelo avisa que un cliente podría irse, qué tan seguido acierta. Mide las alertas innecesarias. Aquí: 22 de 36 alertas fueron abandonos reales.
Recall	De todos los clientes que sí abandonaron, cuántos logró detectar el modelo. Mide los abandonos que se escapan. Aquí: 22 de 35.
F1	Un solo número que combina precisión y recall. Sube cuando ambas son buenas a la vez. Aquí: 0,620 en una escala de 0 a 1.

12. Descripción de cada sección de la aplicación

Panel principal

Muestra una bienvenida breve y cuatro tarjetas: clientes evaluados, clientes que requieren atención alta, clientes con atención baja y promedio de probabilidad de abandono. Incluye cuatro gráficas (clientes por nivel de atención, satisfacción de quienes permanecen y de quienes abandonaron, distribución de edades y abandono según antigüedad) y una tabla con las cinco últimas evaluaciones.

Nueva evaluación

Formulario agrupado en tres bloques (identificación, comportamiento de compra y relación con la empresa), con ayudas breves en cada campo. Al presionar Evaluar cliente se muestra el nombre del cliente, el nivel de atención, la probabilidad estimada, una barra de progreso con los cortes de 40 % y 70 % y una recomendación. También se muestra una comparación con el promedio de la base y la sección Factores que influyeron en esta evaluación, que indica cuánto cambiaría la probabilidad si cada dato tuviera el valor típico. Son asociaciones del modelo y no causas comprobadas.

Nivel	Probabilidad	Recomendación
Bajo	Menor a 40 %	El cliente presenta señales estables. Mantenga el seguimiento habitual.
Medio	De 40 % a 70 %	Se recomienda revisar su experiencia y mantener contacto preventivo.
Alto	70 % o más	Se recomienda priorizar una acción de retención o contacto personalizado.

Historial

Guarda cada evaluación con el nombre, la fecha y hora, los datos principales del cliente, el nivel de atención, la probabilidad y la recomendación. Permite buscar por nombre, filtrar por nivel, descargar un archivo CSV y eliminar el historial, con una confirmación previa.

Análisis técnico

Reúne la información de Machine Learning explicada de forma sencilla: modelo seleccionado, accuracy, cantidad de datos de entrenamiento y prueba, comparación de modelos, matriz de confusión, reporte de clasificación, validación cruzada, significado de las métricas, variables más importantes y limitaciones.

Los niveles de atención se indican siempre con texto además de color, para que la información sea accesible.

13. Flujo de funcionamiento

1. **Usuario:** abre la sección Nueva evaluación.
2. **Formulario:** ingresa los datos del cliente. Si falta el nombre, la aplicación lo advierte.
3. **Preparación de datos:** los valores se organizan en una tabla con las columnas en el mismo orden del entrenamiento.
4. **Modelo:** el Pipeline guardado escala los datos y calcula la probabilidad de abandono con predict_proba.
5. **Resultado visual:** se asigna el nivel de atención (bajo, medio o alto) y se muestran la probabilidad, la barra de progreso, la recomendación y los gráficos.
6. **Registro en el historial:** la evaluación se guarda en SQLite y queda disponible en el historial y en el panel principal.

El modelo se entrena una sola vez con entrenar.py y se guarda en ml/modelo.pkl. La aplicación solo lo carga y predice, no vuelve a entrenar.

14. Capturas de la aplicación

Panel principal

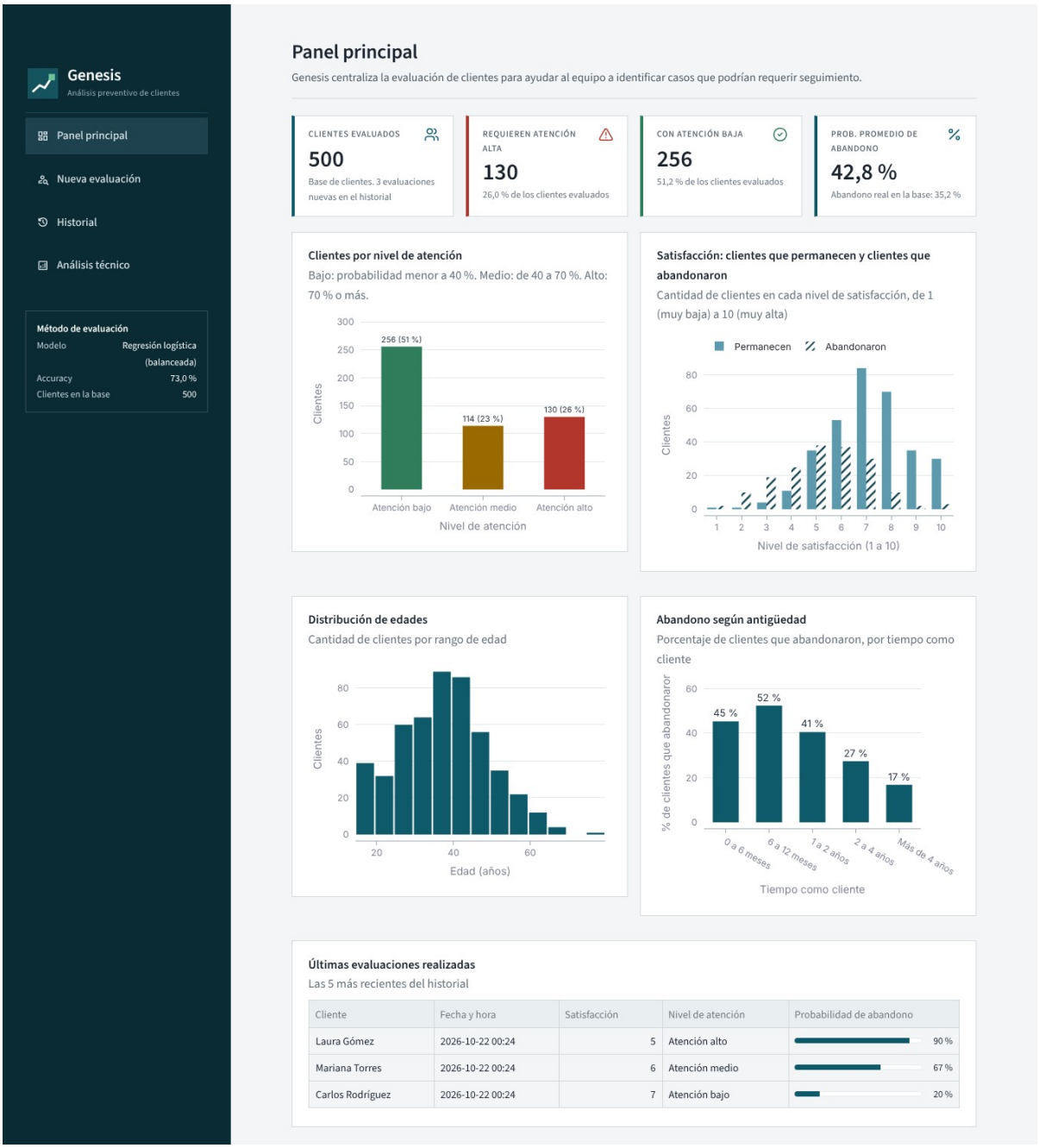


Figura 1. Panel principal en computador

Nueva evaluación

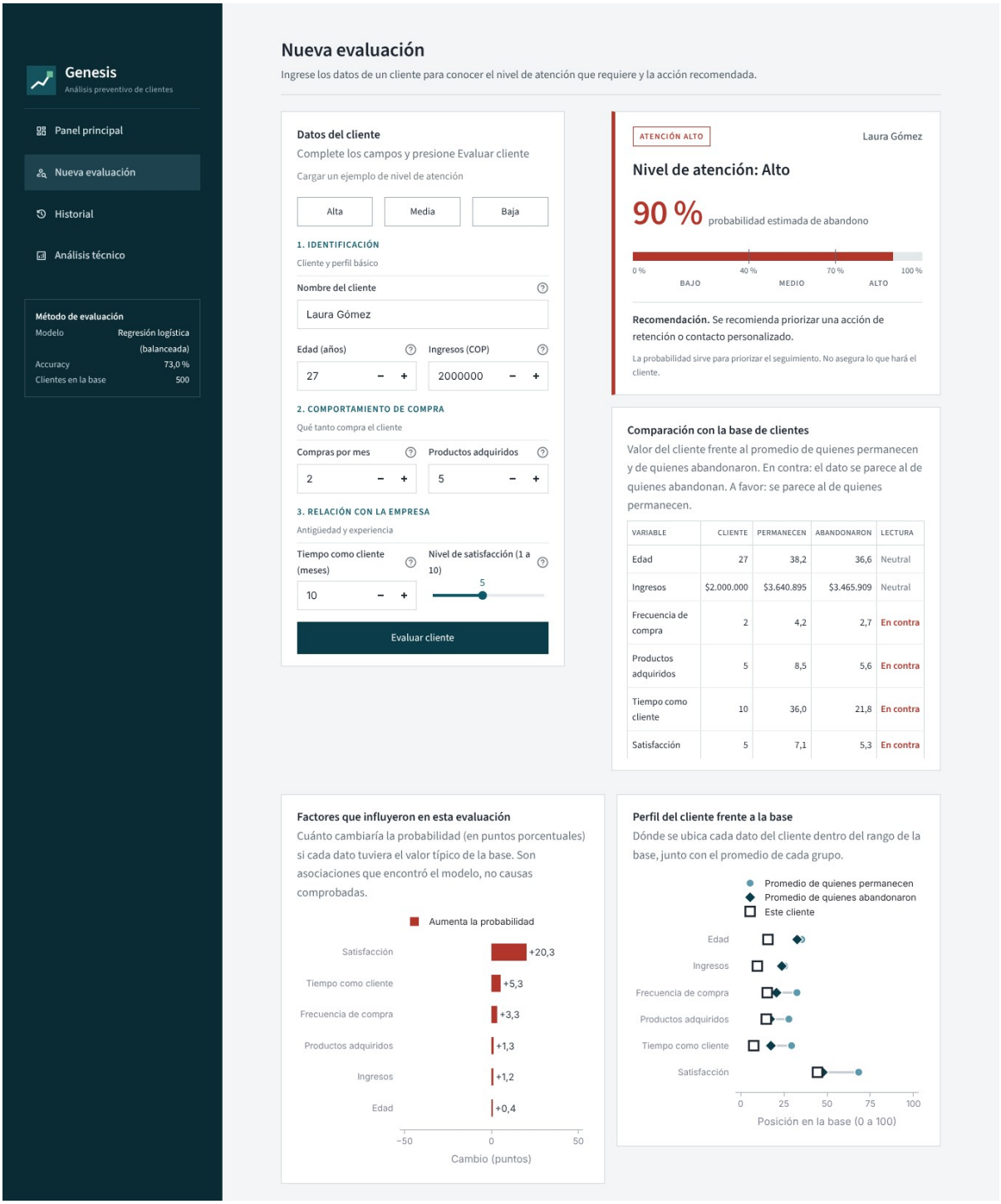


Figura 2. Resultado de una evaluación con atención alta

Historial

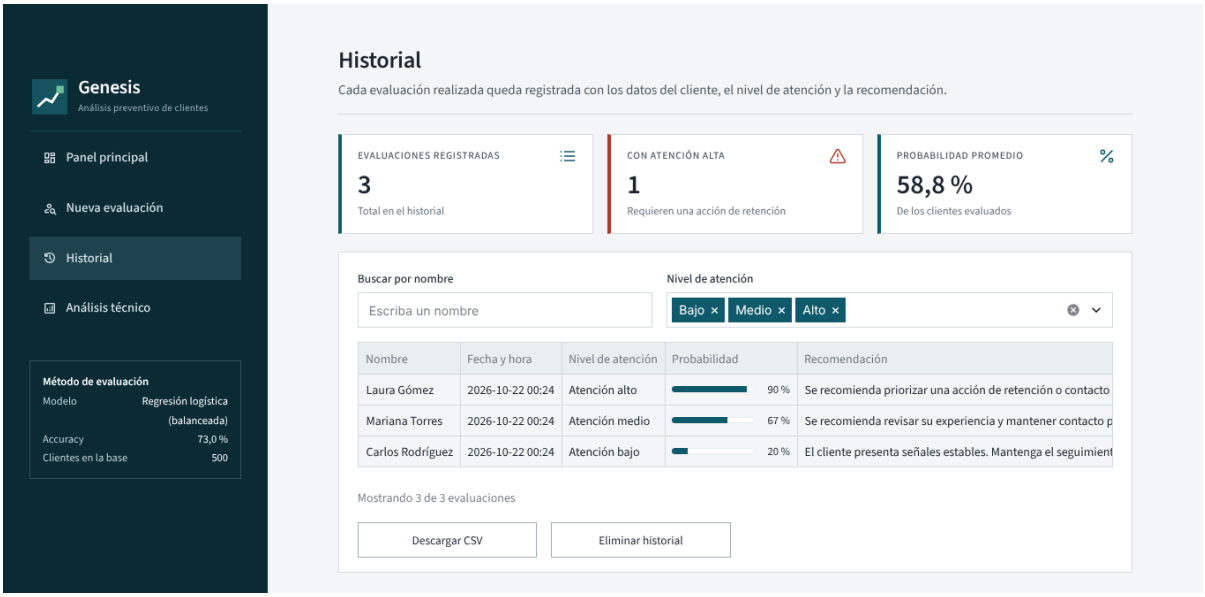


Figura 3. Historial de evaluaciones

Análisis técnico

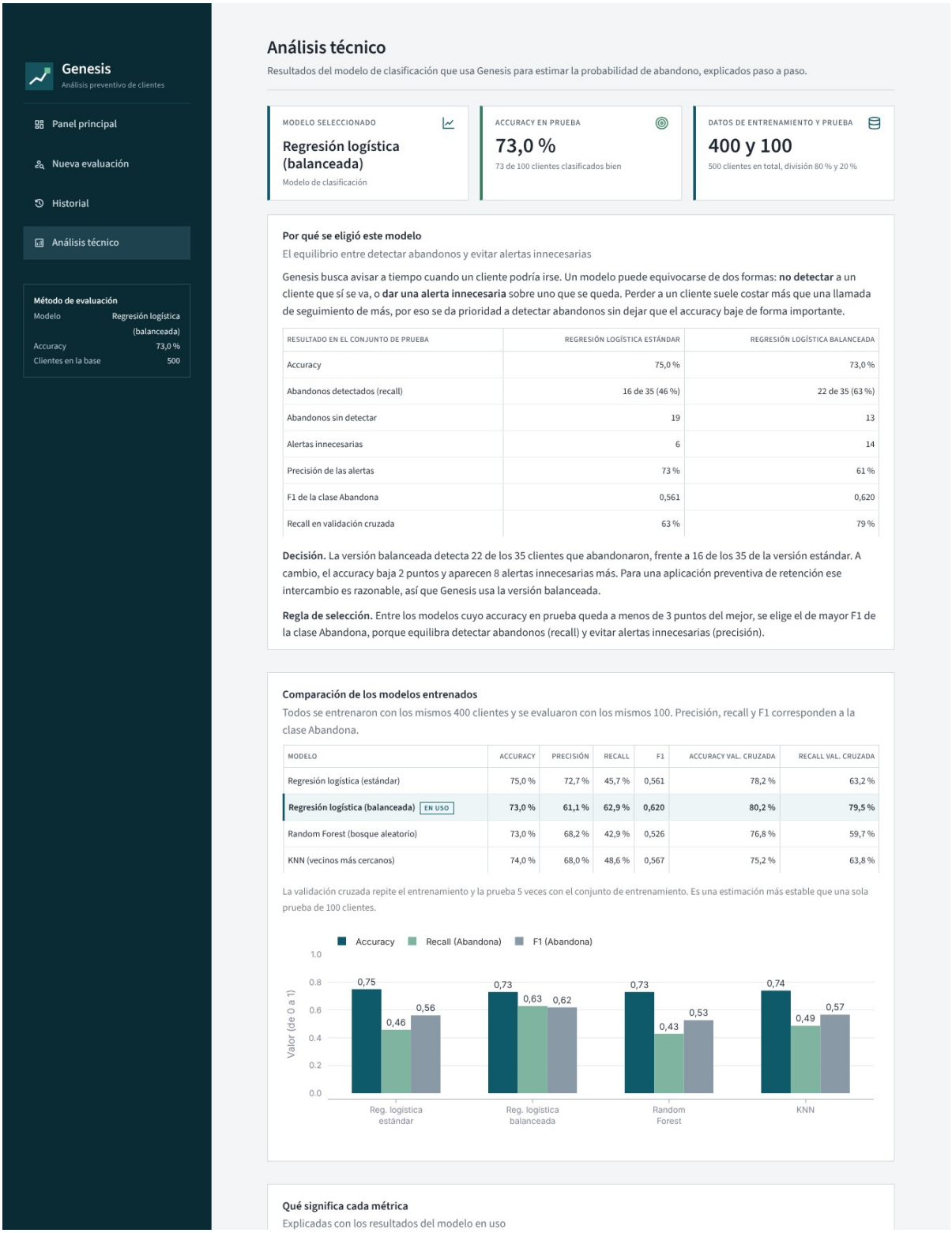


Figura 4. Análisis técnico, primera parte

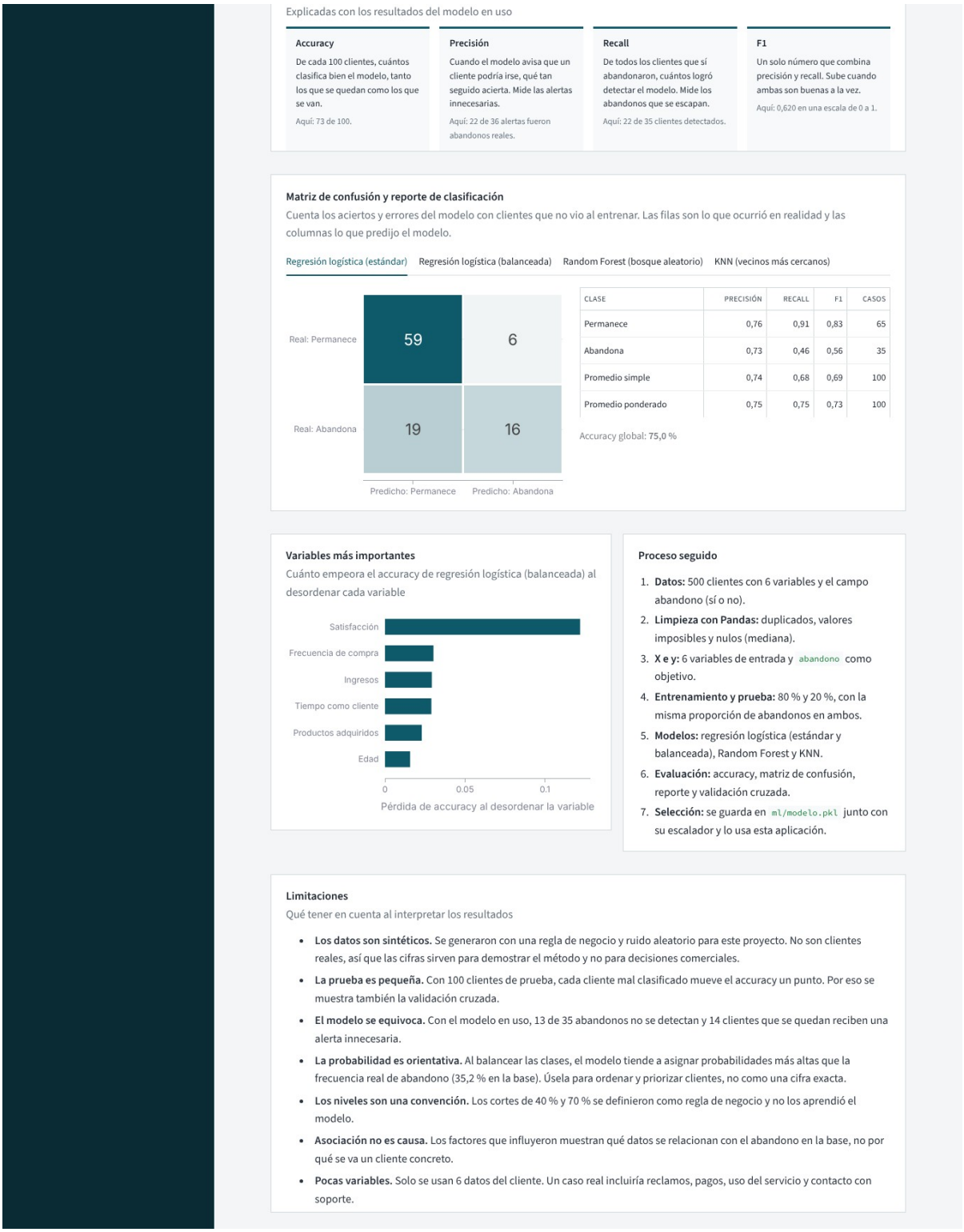


Figura 5. Análisis técnico, segunda parte

Versión para celular

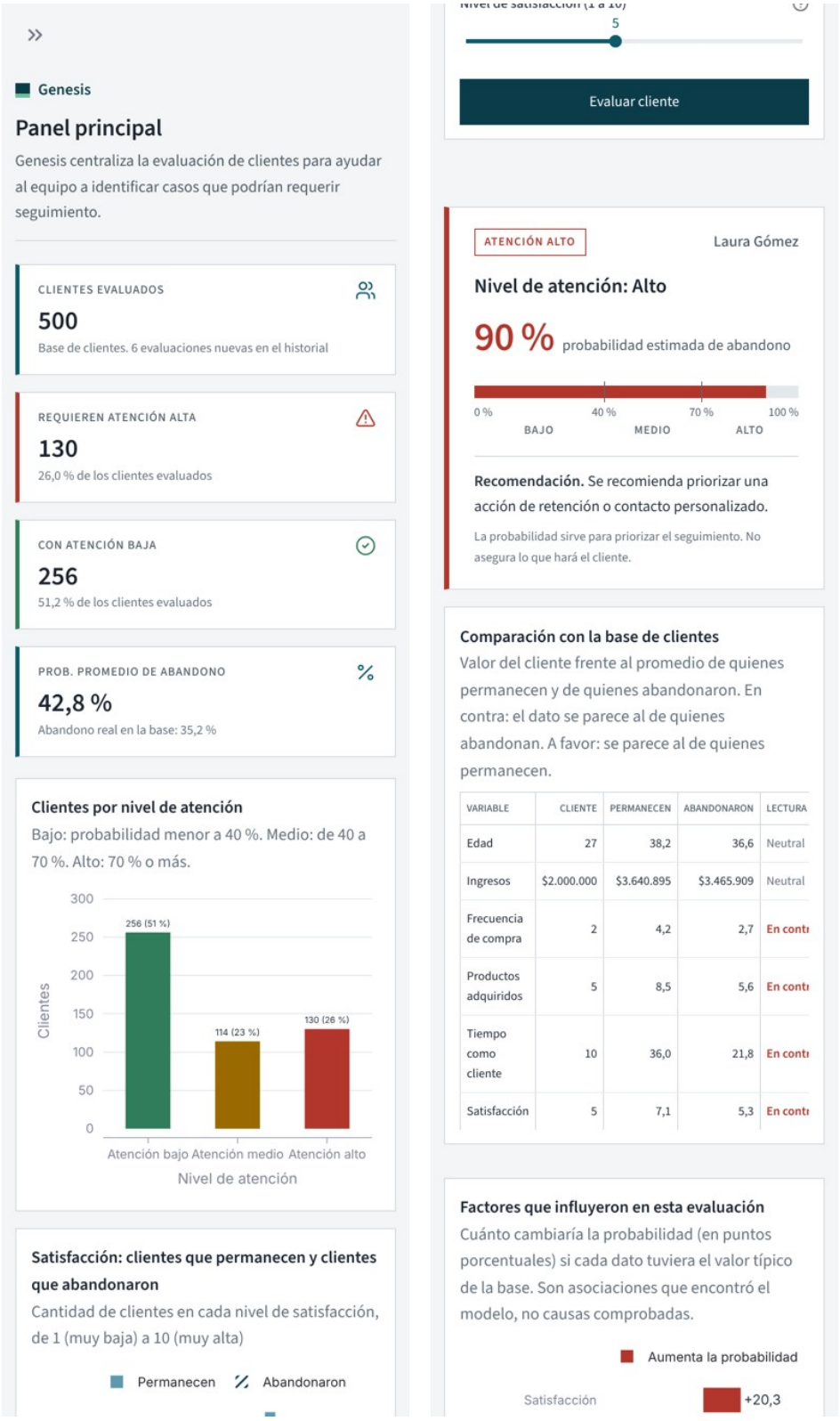


Figura 6. Panel principal y resultado de una evaluación en pantalla de celular

15. Limitaciones del proyecto

- **Los datos son sintéticos.** Se generaron con una regla de negocio y ruido aleatorio. Las cifras sirven para demostrar el método y no para tomar decisiones comerciales reales.
- **La prueba es pequeña.** Con 100 clientes de prueba, cada error mueve el accuracy un punto. Por eso se incluye la validación cruzada.
- **El modelo se equivoca.** Con el modelo en uso, 13 de 35 abandonos no se detectan y 14 clientes que se quedan reciben una alerta innecesaria.
- **La probabilidad es orientativa.** Al balancear las clases, el modelo tiende a dar probabilidades más altas que la frecuencia real de abandono (35,2 % en la base). Sirve para priorizar clientes, no como cifra exacta.
- **Los niveles son una convención.** Los cortes de 40 % y 70 % son una regla de negocio y no los aprendió el modelo.
- **Asociación no es causa.** Los factores que influyeron muestran qué datos se relacionan con el abandono en la base, no por qué se va un cliente concreto.
- **Pocas variables.** Solo se usan seis datos del cliente. Un caso real incluiría reclamos, pagos, uso del servicio y contacto con soporte.
- **Historial temporal en la nube.** SQLite guarda el historial en el disco de la aplicación. En Streamlit Community Cloud ese disco es temporal, por lo que el historial puede reiniciarse. Una base externa resolvería este punto.

16. Conclusiones

- Se construyó una aplicación web completa en Python que cubre todo el recorrido: datos, limpieza, entrenamiento, evaluación de modelos, interfaz, resultado visual e historial.
- Entre cuatro modelos, ninguno fue claramente superior en accuracy (73 % a 75 %), lo que muestra que el criterio de selección importa más que la diferencia de un punto. Se eligió la regresión logística balanceada porque detecta 63 % de los abandonos frente a 46 % de la versión estándar.
- La satisfacción es, con diferencia, la variable más relacionada con el abandono en estos datos, seguida de la frecuencia de compra, los ingresos y el tiempo como cliente.
- Presentar el resultado como nivel de atención con una recomendación hace que el modelo sea útil para personas que no conocen Machine Learning.
- Como trabajo futuro se recomienda usar datos reales, ajustar los cortes de riesgo con el costo real de cada tipo de error y reemplazar SQLite por una base externa para conservar el historial en la nube.

17. Repositorio de GitHub

Código fuente: https://github.com/Ndeahxyzlo/Machine-learning

Si el repositorio cambia de dirección, edite este enlace: seleccione el texto, presione Ctrl+K y pegue la nueva dirección.

18. Despliegue en Streamlit Cloud

Enlace de la aplicación desplegada: [Pegue aquí el enlace de su aplicación en Streamlit Cloud]

Para publicarla: ingrese a share.streamlit.io con su cuenta de GitHub, cree una aplicación nueva, elija este repositorio, la rama main y el archivo principal app.py. Después reemplace el enlace de arriba por la dirección que le asigne Streamlit.

Ejecución local: pip install -r requirements.txt, python entrenar.py (opcional) y streamlit run app.py.